Git Repository:

- Git Repository Structure:

  A Git repository consists primarily of two components inside the .git folder:

  - objects/ — stores all Git objects (blobs, trees, commits)
  - refs/ — stores references (branches, tags)

    Additionally, the .git/HEAD file points to the current branch reference.

- Git Objects:
  - Blob: Stores file contents.
  - Tree: Directory listing referencing blobs and subtrees.
  - Commit: Snapshot pointing to a tree, with metadata and references to parent commits.
- Index (Staging Area):

  An internal file (.git/index) that tracks staged changes by associating file names with blob hashes.
- Branches:

  Files inside .git/refs/heads/ that contain the SHA-1 hash of the commit the branch points to. A branch is essentially a named reference to a commit.

Important Technical Details

- Storage of Objects:

  Git stores objects under .git/objects/ in subfolders named after the first two characters of the SHA-1 hash. This enables efficient lookup even with hundreds of thousands of objects by partitioning the storage.
- Blob Creation:

  Every time a file is staged (git add), Git creates a new blob object for the entire file contents, not just the changes. Even a one-character change results in a new full blob.

- Index File:

  The index is a binary file that tracks the state of the staging area. Adding a file updates this index with the blob SHA and file path.
- Commit Object Structure:

  A commit points to a tree object, includes author/committer metadata, a commit message, and optionally one or more parents (except the first commit).
- Branch as a Reference:

  Branches are simple files containing the SHA-1 of the commit they point to. HEAD is a symbolic reference to a branch.
- Plumbing vs. Porcelain:
  - *Porcelain commands*: High-level, user-friendly commands (git init, git add, git commit, git branch).
  - *Plumbing commands*: Low-level commands used internally or for advanced operations, e.g., git hash-object, git update-index, git write-tree, git commit-tree, git cat-file.

Git Diff and Patch

- A diff shows differences between two files or snapshots, highlighting changes minimally.
- A patch extends diff with contextual information like file names and context lines, enabling broader application of changes.

Git Merge

- Merging combines changes from multiple branches into a single new commit that becomes reachable from all involved branches.
- It complements branching by allowing simultaneous workstreams to be combined later.
- Merging preserves the full history, including branch divergence and convergence.
- The three-way merge algorithm was explained, along with automatic conflict resolution and manual conflict handling.

Git Rebase

- Unlike merge, rebase rewrites history by creating new commit objects based on a new base commit, producing a linear commit history.
- The rebase command changes the base of a branch, replaying commits onto a different point in history.
- This results in a cleaner, more linear project history but alters commit identities.

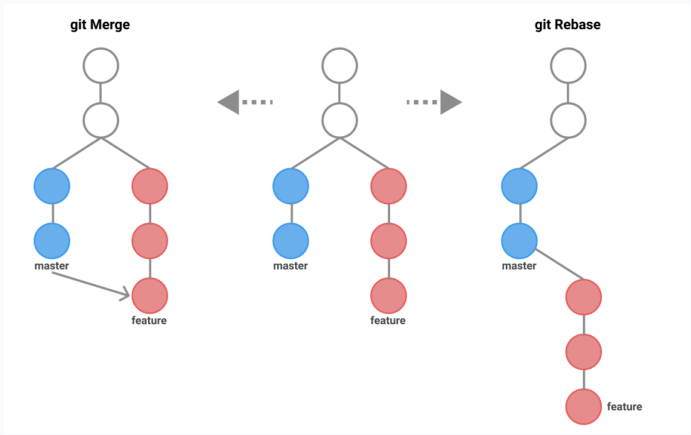
Perspectives on Git History

Two distinct philosophies about Git commit history were discussed:

Perspective	Description	Recommended Practices
Historical Record	Git history as a truthful, immutable record of what actually happened, including mistakes.	Use git revert to undo mistakes without rewriting history. Avoid rewriting pushed commits.
Storytelling for Clarity	Git history as a curated narrative aimed at readability and clarity for collaborators and reviewers.	Use git rebase and git reset to clean and refine commit history before sharing.

When to Use Merge vs. Rebase

- Merge is ideal when preserving the exact history of how branches diverged and converged is important, such as in teams prioritizing historical accuracy.
- Rebase suits workflows emphasizing a clean, linear history for easier code review and comprehension, often used before pushing feature branches.



Git Bisect

Git Bisect automates the process by using a binary search technique to reduce the number of commits to check significantly (from 500+ down to 7-13 steps).

Binary Search: Git Bisect uses binary search to reduce the number of commits you need to test, making the process much more efficient.

Automation: Automating the test procedure within the bisect process can save a lot of time, especially in complex projects

1. Start Bisect Session

```
git bisect start
```

2. Mark a Known Good Commit

```
git bisect good <good-commit-hash>
```

3. Mark a Known Bad Commit

```
git bisect bad <bad-commit-hash>
# or, if the latest commit is bad:
git bisect bad HEAD
```

4. Git Automatically Checks Out the Middle Commit

Run your program/tests on this commit to see if the bug is present.

5. Mark the Commit as Good or Bad

```
git bisect good
```

```
# or
git bisect bad
```

6. Repeat

Git continues moving to the next midpoint.

You test each commit and mark it good or bad.

7. Git Finds the First Bad Commit

Git will display the commit hash that introduced the bug.Git Bisect progressively narrows down the search, checking midpoints between good and bad commits until it finds the first commit that introduced the bug

8. Reset Bisect

```
git bisect reset
```